CASE STUDY 1

Student Record Management System

Java File Handling | Menu-Driven CRUD Application

Technology	Java	**Storage**	Plain-text file: student.txt
Application type	Console-based	**Core operations**	Create, Read, Update, Delete

1. Problem Statement

You are required to develop a Java application to manage student information using a text file named **student.txt**. The file must store a collection of student records. Develop a menu-driven console application that allows the user to add, view, search, update, and delete student records.

2. Student Record Attributes

Each student record must contain the following nine attributes:

Field	Description	Suggested data type
rollNo	Unique student roll number	int
name	Student name	String
age	Student age	int
m1, m2, m3, m4, m5	Marks in five subjects	int (each)
grade	Student grade	String

3. Application Menu

Option	Menu item	Required behaviour
1	Add Record	Accept student details and add a new record to student.txt.
2	Read All Records	Display every student record stored in the file.
3	Read Record by Roll No	Search for and display a student using the roll number.
4	Read Record by Name	Search for and display student record(s) using the name.
5	Delete Record by Roll No	Delete the student record matching the given roll number.
6	Update Record by Roll No	Find a student by the existing roll number and update all other details: name, age, five subject mark
7	Exit	Terminate the application.

4. File Storage Format

Store one student record per line in **student.txt**. Separate the fields using commas in the following order:

```
rollNo,name,age,m1,m2,m3,m4,m5,grade
```

Sample records:

```
101,Rahul,21,85,90,78,88,92,A
102,Arjun,20,75,80,82,79,85,B
103,Kiran,21,90,95,92,88,94,A+
```

Each line represents one student. The field order must remain consistent when writing and reading records. For this simple comma-separated format, student names and grades should not contain commas.

5. Functional Requirements

• The application must repeatedly display the menu until the user selects Exit.

• A roll number must be unique; do not add another record with an existing roll number.

• Read All Records must display all available student records.

• Search by roll number must display the matching record or an appropriate not-found message.

• Search by name should handle matching names; more than one student may have the same name.

• Delete and update operations must identify the target student using the roll number.

• When updating, retain the original roll number and update the student's name, age, five marks, and grade.

• Handle an empty or not-yet-created student.txt file without crashing.

• Handle invalid menu input and file-related exceptions appropriately.

6. Implementation Guidelines

Learning sequence: First implement the application using Strings and String arrays to understand file reading, parsing, searching, updating, and deleting. After completing and testing that version, implement the same functionality using a Student class and objects.

Suggested Java concepts: Scanner, do-while loop, switch statement, methods, FileReader, FileWriter, BufferedReader, BufferedWriter, String.split(), arrays, ArrayList (in the object-based version), and exception handling.

Note: Adding a record can use append mode. Updating or deleting a particular line in a text file generally requires reading the records, modifying the data in memory, and rewriting the file.

7. Expected Deliverables

• Java source code for the menu-driven student management application.

• student.txt containing sample student records.

• A working demonstration of all six CRUD/search menu operations and Exit.

• A short README describing the project, file format, and compilation/run commands.

Objective: Build a file-based student record management application and demonstrate a clear understanding of Java file handling and CRUD operations.